

Encrypted Traffic Classification Through Deep Domain Adaptation Network With Smooth Characteristic Function

Van Tong[✉], Cuong Dao[✉], Hai-Anh Tran[✉], Duc Tran[✉], Huynh Thi Thanh Binh[✉], Thang Hoang-Nam[✉],
and Truong X. Tran[✉], *Senior Member, IEEE*

Abstract—Encrypted network traffic classification has become a critical task with the widespread adoption of protocols such as HTTPS and QUIC. Deep learning-based methods have proven to be effective in identifying traffic patterns, even within encrypted data streams. However, these methods face significant challenges when confronted with new applications that were not part of the original training set. To address this issue, knowledge transfer from existing models is often employed to accommodate novel applications. As the complexity of network traffic increases, particularly at higher protocol layers, the transferability of learned features diminishes due to domain discrepancies. Recent studies have explored Deep Adaptation Networks (DAN) as a solution, which extends deep convolutional neural networks to better adapt to target domains by mitigating these discrepancies. Despite its potential, the computational complexity of discrepancy metrics, such as Maximum Mean Discrepancy, limits DAN's scalability, especially when applied to large datasets. In this paper, we propose a novel DAN architecture that incorporates Smooth Characteristic Functions (SCFs), specifically SCF-unNorm (Unnormalized SCF) and SCF-pInverse (Pseudo-inverse SCF). These functions are designed to enhance feature transferability in task-specific layers, effectively addressing the limitations posed by domain discrepancies and computational complexity. The proposed mechanism provides a means to efficiently handle situations with limited labeled data or entirely unlabeled data for new applications. The aim is to limit the target error by incorporating a domain discrepancy between the source and target distributions along with the source error. Two statistics classes, SCF-unNorm and SCF-pInverse, are used to minimize this domain discrepancy in traffic classification. The experimental results demonstrate that our proposed mechanism outperforms existing benchmarks in terms of accuracy, enabling real-time traffic classification in network systems. Specifically, we achieve up to 99% accuracy with an execution time of only three milliseconds in the considered scenarios.

Index Terms—Traffic classification, deep learning, deep adaptation network, multi-kernel maximum mean discrepancy and smooth characteristic function test.

I. INTRODUCTION

WITH the proliferation of the Internet and Internet of Things (IoT), various applications have emerged, including video streaming, file transfer, and chat services. Additionally, catering to user-specific requirements, such as high reliability and minimal data loss, has become a critical concern [1], [2]. In this landscape, traffic classification, which categorizes network traffic, has assumed paramount importance, especially in the realm of encrypted traffic protocols like VPN, HTTPS and QUIC [3], [4]. The primary aim is to enhance network management capabilities, encompassing Quality of Service (QoS) provisioning, intrusion detection, application-aware mechanisms, firewalling, and more. The diagram of traffic classification is described in Figure 1.

In the past, many solutions have been designed to identify network applications [5], [6], such as port-based classification approaches, deep packet inspection, and statistics-based approaches. However, these approaches suffer from several limitations such as dynamic port allocation, hidden signatures in encrypted traffic, frequent behavior updates in tunneling services and so on. To deal with these limitations, many studies put a special focus on payload-based encrypted traffic classification using Deep Learning (DL) [7], [8], which converts the payload of the packets to byte values and analyzes it using DL to identify the applications. The payload-based approaches focus on ‘existing applications’, which contain labeled data in the training dataset. In detail, this approach collects a large amount of labeled application data and trains the learning models to identify these applications. However, these approaches encounter challenges with ‘new applications’, which are not in the training dataset and contain unlabeled data or limited labeled data. Concretely, manual labelling is limited due to being time-consuming and costly. With the emergence of Transfer Learning (TL), many recent studies illustrate that DL algorithms can learn implicit features from an original domain (e.g., existing applications, etc.) and transfer it to a new domain (e.g., new applications, etc.) in order to identify new applications with limited labeled data [9]. In TL algorithms, the parameters in the task-specific layers cannot be learned effectively due to the increase in domain discrepancy during

Received 15 November 2023; revised 1 April 2024 and 19 September 2024; accepted 24 January 2025. Date of publication 27 January 2025; date of current version 14 March 2025. This research was funded by Vingroup Innovation Foundation (VINIF) under project code VINIF.2022.DA00183 for Huynh Thi Thanh Binh and Hai-Anh Tran. The associate editor coordinating the review of this article and approving it for publication was Y. Diao. (Corresponding author: Hai-Anh Tran.)

Van Tong, Hai-Anh Tran, Duc Tran, and Huynh Thi Thanh Binh are with the School of Information and Communication Technology, Hanoi University of Science and Technology, Hanoi 10000, Vietnam (e-mail: vantv@soict.hust.edu.vn; anhth@soict.hust.edu.vn; ductq@soict.hust.edu.vn; binhht@soict.hust.edu.vn).

Cuong Dao and Thang Hoang-Nam are with the Faculty of Information Technology, Hanoi University of Civil Engineering, Hanoi 11600, Vietnam (e-mail: cuong28264@huce.edu.vn; thanghn@huce.edu.vn).

Truong X. Tran is with the School of Science, Engineering and Technology, Penn State Harrisburg, The Pennsylvania State University, University Park, PA 16802 USA (e-mail: truong.tran@psu.edu).

Digital Object Identifier 10.1109/TNSM.2025.3534791

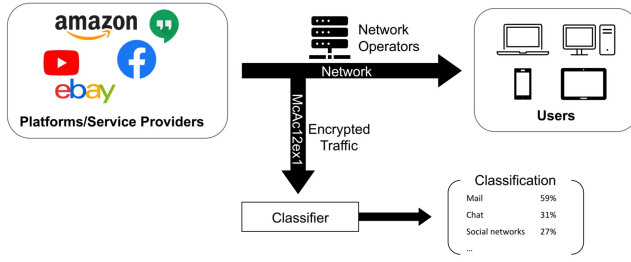


Fig. 1. The diagram of traffic classification.

the feature transferability from the original domain to the new domain [9], [10]. Hence, it is essential to reduce the dataset bias and improve the feature transferability to adapt to new applications.

To address the decrease in feature transferability, numerous studies have investigated the Deep Adaptation Network (DAN) architecture in the context of traffic classification, exploring methods such as Maximum Mean Discrepancy (MMD) [11], [12], [13], semi-supervised learning [14], and others. While achieving good performance in the considered scenarios, the usage of MMD comes with several limitations [15]. Firstly, MMD employs a smooth witness function, which proves to be less effective in high-dimensional spaces. Additionally, relying on the witness function does not maximize test power, as it fails to provide a direct indication of distributional features. In statistics, the power of a hypothesis test denotes the probability of correctly rejecting the null hypothesis, representing the likelihood of true positive detection. Secondly, MMD-based approaches demand significant training time when dealing with large input datasets, attributed to the computational complexity of calculating MMD between the source and target domains.

To overcome these limitations, we propose a novel encrypted traffic classification mechanism to identify new applications using DAN architecture and novel Smooth Characteristic Functions (SCFs): **SCF-unNorm** (Unnormalized SCF) and **SCF-pInverse** (Pseudo-inverse SCF). These characteristic functions are built on the analytic representations of probability distributions that have greater power than MMD. Moreover, the time complexity of characteristic functions is lower than MMD. The proposed mechanism offers opportunities to deal effectively with limited labeled data or unlabeled data of new applications. Consider a use case where the Internet is divided into numerous Autonomous Systems (ASs), each managed by an Internet Service Provider (ISP). ISPs need to implement traffic classification approaches to support network management. In the case where there is a new AS with limited labeled data, or unlabeled data of novel applications which did not exist in other ASs, using knowledge (pre-trained models) from existing ASs to identify novel applications leads to inaccurate classification results. Therefore, the proposed mechanism helps to enhance the performance of encrypted traffic classification in the context of limited labeled data or unlabeled data.

This work is an extended version of our previous paper [16]. Our previous efforts concentrated on employing DAN with

MMD and Mean Embedding (ME) for encrypted traffic classification. However, these approaches were hampered by extensive training durations. Furthermore, the experimental results presented in that study were incomplete, indicating that the proposed solution lacked efficacy. Consequently, in this paper, we expand upon this research by introducing a deep domain adaptation network incorporating novel SCFs to overcome these shortcomings. The contributions of the paper are presented as follows:

- *Collection of Encrypted Traffic Dataset for Domain Adaptation:* This work introduces *DAQUIC*, a dataset specifically designed for domain adaptation in encrypted traffic classification, addressing the lack of publicly available datasets. The dataset is accessible on GitHub [17].
- *Payload-based Encrypted Traffic Classification with a Balance Between Accuracy and Processing Time:* A Deep Learning-based encrypted traffic classification mechanism is proposed, offering high accuracy while maintaining efficient processing times.
- *DAN-based Encrypted Traffic Classification Using Novel SCFs:* To reduce domain discrepancies in task-specific layers, a DAN-based encrypted traffic classification method using novel Smooth Characteristic Functions (**SCF-unNorm** and **SCF-pInverse**) is proposed.

The rest of the paper is structured as follows. Section II introduces related work in encrypted traffic classification and existing studies using Transfer Learning. In Section III, the paper presents the formalization of the problem related to traffic classification. Section IV illustrates the methodology of the proposed payload-based encrypted traffic classification algorithm, the proposed DAN-based solution, and the domain adaptation networks using two-sample test statistics: SCF-unNorm, SCF-pInverse, and MK-MMD. Section V describes the experimental results of the proposed mechanism for both existing and new applications. Finally, the paper concludes with Section VI, which highlights our future work.

II. RELATED WORK

This section presents an overview of studies on encrypted traffic classification and the use of TL in this domain.

A. Encrypted Traffic Classification

According to [3], [18], there are three popular encrypted traffic classification mechanisms: approaches using Transport Layer Security (TLS) features, statistic-based approaches, and payload-based approaches.

The TLS-based traffic classification mechanism takes into account TLS-specific features extracted from the TLS handshake in the first few packets of a flow. The header of TLS packets contains useful information to identify the applications. Cui et al. proposed “Only Header” [19], which combines a segmentation mechanism and Capsule Neural Networks to classify traffic while avoiding privacy risks. Similarly, He et al. proposed DSCU [20], a skew-based classification approach for classifying TLS applications that analyzes the window-based segmentation using a one-class classifier to identify unknown traffic from the known ones. However, these approaches suffer

from several limitations. “Only Header” may miss critical payload information, which is often encrypted in TLS, leading to potential misclassification, while DSCU falters with the variability and encryption of TLS traffic, causing difficulties in accurately classifying new traffic patterns.

The statistic-based mechanism conveys the first several packets or all packets of a network flow to extract flow-based features (e.g., packet length, inter-arrival time, etc.) for traffic classification. Fathi-Kazerooni et al. [21] proposed a traffic classification mechanism using the Random Forest algorithm to identify six kinds of applications. This mechanism takes into account ten flow-based features from all packets of flows and analyzes them to determine Chrome, WhatsApp, OneNote, Spotify, Google Drive, and OneDrive. However, traffic classification mechanisms using flow-based features from all packets of a flow cannot early identify the network applications because it is required to gather information after a flow terminates. Therefore, many studies focus on traffic classification approaches that consider flow-based features from the first several packets of a flow to early identify the applications. Qin et al. proposed HFC [22], a method for unsupervised traffic classification by exploring the flow features and link patterns. This method formulates the traffic identification task as a graph co-clustering problem with topology and edge attributes and leverages both features and linkage information to classify traffic. Moreover, Shaheem et al. presented a classification technique using an ensemble of deep learning architectures on packet, payload, and inter-arrival time sequences [23]. This approach analyzes the high-level features from network data using ML algorithms. Despite achieving good performance, these approaches require human intervention to identify pre-defined flow-based features. Besides, they are ineffective when application behaviors change with additional services (e.g., Virtual Private Network, etc.).

The payload-based mechanism considers the byte value in the payload of a packet within a flow to identify the network applications. Lotfollahi et al. [8] proposed a novel traffic classification using the DL algorithm called Deep Packet. This approach takes into account 1500 bytes in the payload of a packet and analyzes them to classify the network traffic into different kinds of applications (e.g., email, chat, torrent, etc.) in the context of VPN (Virtual Private Network) and non-VPN. Deep Packet uses all information in the header of a packet, so this can lead to inaccurate classification because many applications use the same port numbers (e.g., port 443, etc.). Wang et al. [24] proposed an encrypted traffic classification approach to identify 15 kinds of applications (e.g., Gmail, Facebook, etc.) for home networks. This mechanism considers the first 1480 bytes of a packet and analyzes it using different learning algorithms such as Multi-layer Perceptron, Stacked Auto-encoder, and so on. Using a large number of byte values leads to high processing time, which is not appropriate for real-time classification, so Wang et al. [7] proposed another traffic classification approach which only uses the first 784 bytes of a packet to identify six VPN and non-VPN applications. Although these payload-based mechanisms can achieve good performance, they require a long processing time due to using

a large number of byte values in the input. However, the trade-off between performance and time processing is not considered thoroughly in existing studies.

While numerous payload-based approaches exist for identifying network applications, they prove ineffective when encountering novel applications that have not been previously trained, leading to inaccurate results. This presents significant challenges for network management.

B. Traffic Classification Using Transfer Learning

According to a recent study [25], there are many solutions for encrypted traffic classification, such as Federated Learning (FL), TL, and so on. FL requires labeled data from network applications to identify them, but it is difficult to obtain labeled data of novel emerging applications. In this context, many TL-based approaches are designed to identify novel applications.

Taghiyarrenani and Farsi [13] proposed a domain adaptation mechanism for traffic classification, which trains a model using labeled samples from a network and adapts it to another network with only a few labeled samples using MMD. Moreover, Ning et al. [14] proposed a novel traffic classification approach based on domain adaptation and semi-supervised learning. This approach generates a model using a semi-supervised learning algorithm from a large amount of unlabeled data and a few labeled samples. Then, this approach adapts the pre-trained model from the source to the target domain using MMD. However, using MMD to calculate the domain discrepancy between the source and the target domains does not maximize the test power and requires high training time.

Similarly, Gu et al. [26] proposed Zen-tor, a zero-knowledge known-unknown traffic classification mechanism using GAN and Convolutional Neural Network (CNN) for home networks. Zen-tor considers labeled known flow samples and analyzes them to train Flow Discriminator D and Fake Flow Generator G so that Zen-tor can classify the new network traffic from the existing one without any prior information about new traffic. The objective is to protect the privacy of the home network with only labeled existing traffic. Similarly, Hu et al. [18] proposed ZSL, a novel attribute-based zero-shot learning framework for traffic classification. ZSL generates a feature-attribute embedding model using both flow features and attributes to identify different kinds of existing applications. After that, ZSL generates FAE-G, a GAN-based feature generation model, to synthesize traffic samples of new applications from the existing ones. Finally, the traffic samples of both existing and new applications are fed into a two-step classification method using gradient-based rejection to identify different kinds of applications. ZSL can achieve good performance for existing applications with over 95%, but it has not good performance for new applications with approximately 21%. Besides, Nukavarapu et al. [27] proposed MirageNet, a GAN-based synthetic network traffic generation framework to generate synthetic models of protocols, applications, and devices. The GAN-based traffic classification mechanisms offer opportunities to classify new traffic from existing ones

without any knowledge about new traffic. However, the classification performance for new traffic is limited. FixMatch [28] is a semi-supervised learning that considers unlabeled data to improve the classification. FixMatch generates pseudo-labels for weakly-augmented unlabeled data via a classification model. Then, it uses the pseudo-labels with high confidence to train the model with strongly-augmented data. Despite achieving good performance in considered scenarios, FixMatch suffers from limitations with the misclassification of pseudo-labels. Minimum Class Confusion (MCC) [29] is a novel general loss designed to adapt to various domain adaption scenarios such as closed-set, partial-set domain adaptation and so on. MCC can achieve other approaches in considered scenarios, particularly the DomainNet dataset. However, MCC is ineffective with imbalanced data when the number of samples in the source domain is significantly higher than in the target domain.

Numerous existing studies incorporate TL for traffic classification, employing methods such as MMD, GAN, etc. However, these approaches encounter challenges related to accuracy limitations and substantial computational complexity. While the MMD approach demonstrates commendable performance under specific circumstances, it often prolongs training durations. Conversely, the GAN approach proves inadequate for handling novel applications, resulting in inaccuracies. Hence, the proposed DAN-based traffic classification mechanism strives to achieve optimal performance with new applications while maintaining reasonable training times.

III. TRAFFIC CLASSIFICATION PROBLEM

Encrypted traffic classification identifies the network traffic generated from different applications and categorizes it into various categories (e.g., video streaming, e-commerce, etc.). The proposed mechanism considers byte-level features from the first few packets of a flow to classify the network traffic into different categories. The traffic classification using the byte values in the payload can be effective with existing domain data (e.g., existing applications, etc.). However, there is insufficient data to train the learning models for the new domains (e.g., new applications, etc.). The proposed method considers both existing and new applications.

For the existing applications that contain labeled data in the training dataset, consider set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ of n samples, where x_i is the byte-level features of the i -th samples and y_i is the category of the i -th samples. Byte-level features of a sample are described in Equation (1):

$$x_i = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1k} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mk} \end{pmatrix} \quad (1)$$

where k is the number of byte values used in the payload of a packet within a flow ($\leq$ maximum byte of a packet), m is the number of consecutive packets used in a flow, b_m^i is the i -th byte of the m -th packet in a flow and byte $x00 \leq b_i \leq xff$. The objective of traffic classification is to build a deep neural network $y = \theta(x)$ which can minimize the target risk $\epsilon(\theta) = Pr[\theta(x) \neq y]$.

As for the new applications not in the training dataset, an unsupervised domain adaptation is utilized to classify them into different categories. Denote by $\mathcal{D}_s = \{(x_i^s, y_i^s)\}_{i=1}^{n_s}$ a source domain with n_s labeled samples, and denote by $\mathcal{D}_t = \{(x_i^t, y_i^t)\}_{i=1}^{n_t}$ a target domain with n_t samples. The target domain $\mathcal{D}_t$ is composed of both labeled data $\mathcal{D}_l = \{(x_i^l, y_i^l)\}_{i=1}^{n_l}$ and unlabeled data $\mathcal{D}_u = \{(x_i^u)\}_{i=1}^{n_u}$. A few labeled samples are required for the new applications, so $\mathcal{D}_l \ll \mathcal{D}_u$. The total dataset is all the available samples $\mathcal{D} = \mathcal{D}_s \cup \mathcal{D}_t$. The source and target domains are related but have different distributions. The probability distribution of the source domain and target domain are p and q , respectively. Besides, p is different from q . The goal is to learn a deep neural network $y = f(x)$ which can learn transferable features between different domains to reduce the discrepancy between source and target domains and minimize the target risk $R(f) = Pr_{(x,y) \sim q}[f(x) \neq y]$. In the considered experiments, the number of bytes k in the payload is set to 256 or 512 while the number of packets in a flow m is set to 20. The number of samples in the dataset is described in Table II. The details are analyzed in the experimental results.

IV. METHODOLOGY

This section proposes novel two-sample test statistics (**SCF-unNorm** and **SCF-pInverse**) in DAN, the payload-based mechanism, and the DAN-based mechanism to classify the encrypted traffic.

A. Novel Sample Statistic Functions and Comparative Benchmark

The domain adaptation is limited when the target domain contains no prior information or limited information about new applications. According to related work [9], [30], there are many studies aiming to bound the target error by adding a domain discrepancy between the source and target distribution p and q with the source error. The two-sample testing refers to which accepts or rejects the null hypothesis $p = q$ using two samples generated from these distributions. In this case, three statistics classes are considered: SCF-unNorm, SCF-pInverse, and MK-MMD [9], [31]. With the large input sequences of traffic classification, calculating MK-MMD leads to a high computational time, influencing the training time. Hence, **SCF-unNorm** and **SCF-pInverse** are proposed to reduce the domain discrepancy in traffic classification.

1) *Novel Smooth Characteristic Functions*: SCF [15] is a kind of test statistic that uses a form of Hotelling's T-squared statistic. The detail of SCF is described in Equation (2):

$$M_k(\mathcal{D}_s, \mathcal{D}_t) \triangleq \max_v n \bar{z}_n^\top S_n^{-1} \bar{z}_n. \quad (2)$$

where $\mathcal{D}_s$ is the source domain data, $\mathcal{D}_t$ is the target domain data, $\bar{z}_n \triangleq \frac{1}{n} \sum_{i=1}^n z_i$, $S_n \triangleq \frac{1}{n-1} \sum_{i=1}^n (z_i - \bar{z}_n)(z_i - \bar{z}_n)^\top$, $z_i := [\hat{f}(\phi(x_i^s)) \sin(\phi(x_i^s)^\top v_j) - \hat{f}(\phi(x_i^t)) \sin(\phi(x_i^t)^\top v_j), \hat{f}(\phi(x_i^s)) \cos(\phi(x_i^s)^\top v_j) - \hat{f}(\phi(x_i^t)) \cos(\phi(x_i^t)^\top v_j)]_{j=1}^J \in \mathbb{R}^{2J}$, $\hat{f} = \int_{\mathbb{R}^d} \exp(-iu^\top x) f(u) du$ denotes the Fourier transform of $f(x)$ and $f: \mathbb{R}^d \rightarrow \mathbb{R}$ is an analytic translation-invariant kernel. ϕ is a nonlinear feature mapping function of hidden representations. SCF relies on J test locations $\mathcal{V} =$

$\{v_j\}_{j=1}^J \subset \mathbb{R}^d$, which are in frequency domain. Let $\psi_p(v) = \int_{\mathbb{R}^d} \varphi_p(w) l(v-w) dw$ be the smooth characteristic function of p where $\varphi_p(w)$ is the characteristic function of p . The SCF is considered as a normalized distance of empirical $\varphi_p(v)$ and $\varphi_q(v)$ by using S_n^{-1} .

In the payload-based traffic classification mechanism, zero padding in the input data results in a singular matrix, so it is difficult to measure the covariance matrix. SCF is composed of an inverse of a matrix S_n^{-1} described as in Equation (3):

$$S_n^{-1} = \frac{S_n^\top}{\det S_n} \quad (3)$$

where $S_n = (a_{ij})_{n \times n}$, $S_n^\top$ is the adjoint matrix of S_n and $\det S_n$ is the determinant of S_n .

Following the Leibniz formula [32], $\det S_n$ is depicted as in Equation (4):

$$\det S_n = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n a_{i, \sigma(i)} \quad (4)$$

In several cases, $\det S_n$ is zero, so the inverse of a matrix S_n^{-1} is undefined. Therefore, two customizations of SCF is proposed to deal with the problem of the singular matrix: **SCF-unNorm** and **SCF-pInverse**.

The **SCF-unNorm** refers to the unnormalized SCF which skips S_n^{-1} . Besides, **SCF-unNorm** is demonstrated as a metric on the space of probability measures for any $\mathcal{V}$ and to act for two-sample testing in a manner comparable to the normalized SCF [33]. The SCF without using S_n^{-1} does not require considering the covariance matrix. The detail of **SCF-unNorm** is depicted as in Equation (5):

$$M_k(\mathcal{D}_s, \mathcal{D}_t) \triangleq \max_v n \bar{z}_n^\top \bar{z}_n. \quad (5)$$

SCF-pInverse considers the Moore–Penrose pseudo-inverse [34] instead of the inverse matrix S_n^{-1} in Equation (2). The detail of **SCF-pInverse** is depicted as in Equation (6):

$$M_k(\mathcal{D}_s, \mathcal{D}_t) \triangleq \max_v n \bar{z}_n^\top S_n^+ \bar{z}_n. \quad (6)$$

where S_n^+ is the Moore–Penrose inverse of matrix S_n . The Moore–Penrose inverse of the matrix S_n is described as in Equation (7):

$$S_n^+ = \begin{cases} (S_n^\top S_n)^{-1} S_n^\top, & \text{if } S_n \text{ has linearly independent columns.} \\ S_n^\top (S_n S_n^\top)^{-1}, & \text{if } S_n \text{ has linearly independent rows.} \end{cases} \quad (7)$$

2) *Comparative Benchmark*: MMD [9], [31] is a test which minimizes the Type-II error and maximizes the two-sample test power (e.g., rejecting a false null hypothesis, etc.). Let assume that H_k is the reproduction of kernel Hilbert space (RKHS) possessing a distinctive kernel k . The mean embedding of distribution q in H_k is a distinct element $\mu_k(p)$ where $\mathbb{E}_{x \sim p} f(x) = \langle f(x), \mu_k(p) \rangle_{H_k}$ with all $f(x) \in H_k$. MK-MMD between probability distributions p and q is considered as the RKHS distance between the mean embeddings of p and q . The formulation of MK-MMD is described as Equation (8):

$$M_k(p, q) \triangleq \|\mathbb{E}_p[\phi(x^s)] - \mathbb{E}_q[\phi(x^t)]\|_{H_k}^2. \quad (8)$$

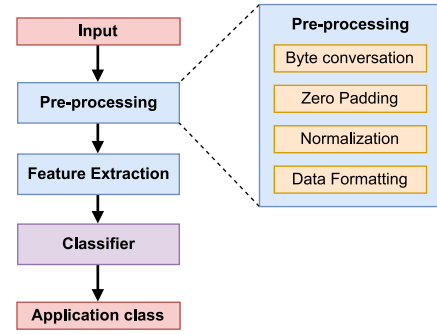


Fig. 2. The encrypted traffic classification architecture.

where ϕ is a nonlinear feature mapping of hidden representations.

$\mathcal{D}_s$ and $\mathcal{D}_t$ are the samples from the distributions p and q . The formulation of MK-MMD is converted to Equation (9):

$$\begin{aligned} M_k(\mathcal{D}_s, \mathcal{D}_t) &\triangleq \frac{1}{n_s^2} \sum_{i=1}^{n_s} \sum_{j=1}^{n_s} k(x_i^s, x_j^s) \\ &+ \frac{1}{n_t^2} \sum_{i=1}^{n_t} \sum_{j=1}^{n_t} k(x_i^t, x_j^t) \\ &- \frac{2}{n_s n_t} \sum_{i=1}^{n_s} \sum_{j=1}^{n_t} k(x_i^s, x_j^t). \end{aligned} \quad (9)$$

A noticeable feature from Equation (8) is that $p=q$ if $M_k(p, q) = 0$ [31]. The characteristic kernel $k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$ is considered as the convex combination of m positive-definite kernels k_u :

$$\mathbb{K} \triangleq \left\{ k = \sum_{u=1}^m \beta_u k_u : \sum_{u=1}^m \beta_u = 1, \beta_u \geq 0 \forall u \right\}. \quad (10)$$

where the constraints on coefficients β_u are required to ensure that the multi-kernel k is characteristic.

To guarantee the test power and minimal test error, the kernel chosen for the mean embeddings of p and q is essential. The multi-kernel k can use various kernels to improve the MK-MMD, resulting in a principled technique for the optimal kernel selection.

B. Payload-Based Encrypted Traffic Classification Approach

The payload-based mechanism aims to analyze the network traffic and categorize it into different applications. This mechanism contains two main stages: pre-processing and feature extraction. As illustrated in Figure 2, in the beginning, the data traffic is processed in the pre-processing to create a matrix representing a network flow. After that, this matrix is fed into a feature extractor to classify the network traffic into different categories: e-commerce, video on-demand, and interactive data.

In the pre-processing stage, the network traffic is processed to create uniform data for the feature extraction. This stage comprises four steps: byte conversation, zero padding, normalization, and data formatting. The proposed traffic classification mechanism takes into account the payload of each packet in

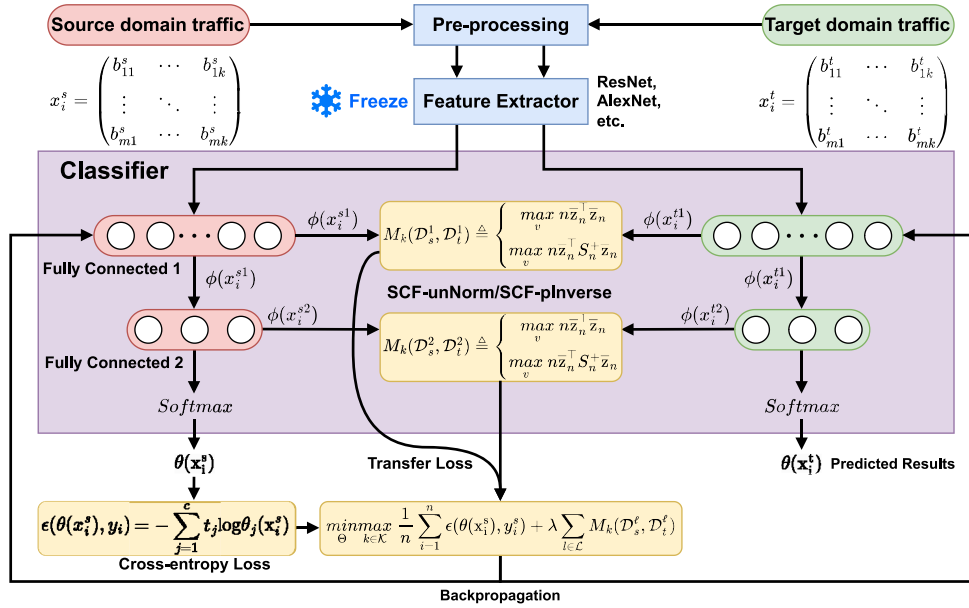


Fig. 3. The DAN-based encrypted traffic classification mechanism using SCF.

a flow for the traffic classification, so the other information (e.g., data-link header, Ethernet header, etc.) is skipped due to an unnecessary [8]. Then, the bit value in the payload is converted to byte values to reduce the input size in the byte conversation. The proposed payload-based mechanism considers m first bytes in the payload of packets. If the number of selected bytes is m , and the payload length of a packet is less than m , the payload is padded zero values at the end to make an equal length in the normalization. To enhance the performance, the byte values are then normalized into a value from 0 to 1 in the normalization. To represent a network flow, the payload of the first k consecutive packets within a flow are grouped to create a 2-dimensional vector as in the existing study [35].

After processing the network traffic, this 2-dimensional vector is analyzed by a feature extractor (e.g., ResNet50, DarkNet53, etc.) to classify the network traffic into different types of applications (e.g., YouTube, VoIP, Chat, Amazon, etc.). To adapt to the traffic classification, the fully-connected layer of the feature extractor is replaced by two other fully-connected layers. Then, the output from these layers is put into the softmax function, which normalizes the output to a probability distribution over predicted applications.

C. DAN-Based Encrypted Traffic Classification Mechanism

In order to enhance the performance of traffic classification, the proposed payload-based mechanism aims to minimize the target risk which is defined in Equation (11):

$$\min_{\Theta} \frac{1}{n} \sum_{i=1}^n \epsilon(\theta(x_i), y_i), \quad (11)$$

where $\theta(x_i)$ is the conditional probability that the classifier assign x_i to all labels, and ϵ is the cross-entropy loss function

which is depicted as Equation (12):

$$\epsilon(\theta(x_i), y_i) = - \sum_{j=1}^c t_j \log \theta_j(x_i), \quad (12)$$

where $\theta_j(x_i)$ is the softmax activation function on the final fully-connected layer, which computes the probability of assigning point x_i to class j , c is the number of classes, and t_j is a truth label taking the value of 0 or 1. If y_i equals the actual label, t_j is set to 1. Otherwise, t_j is set to 0.

In domain adaptation, it is challenging to transfer knowledge from a source domain to a target one because the target domain contains no information or a few labeled data on new applications. Therefore, it is prone to over-fitting or difficult to fine-tune the pre-trained models. It is due to the domain discrepancy between the source and target domains. As a result, this leads to a reduction of feature transferability in task-specific layers (e.g., fully-connected layers). Therefore, in this section, a DAN-based mechanism is proposed to overcome these limitations. Concretely, in the deep adaptation network, we take into account the Smooth Characteristic Function with two modifications in the inverse of the covariance matrix (SCF-unNorm and SCF-pInverse) to deal with the problem of the singular matrix. In the domain adaptation, the distribution of source and target domains are not too different. In this context, the proposed DAN-based solution aims to make the distribution of source and target domains nearly similar in the fully-connected layers. Consequently, a multi-layer penalty is added to the target risk, which is described in Equation (13):

$$\min_{\Theta} \max_{k \in \mathcal{K}} \frac{1}{n} \sum_{i=1}^n \epsilon(\theta(x_i), y_i) + \lambda \sum_{l \in \mathcal{L}} M_k(\mathcal{D}_s, \mathcal{D}_t), \quad (13)$$

where $\mathcal{L}$ represents the list of hidden layers applying the multi-layer penalty for the source and target domains, and k is the index of the hidden layer to which the penalty is

applied. λ is a trade-off parameter, and M_k is the SCF between $\mathcal{D}_s$ and $\mathcal{D}_t$. Denote $\mathcal{D}_s = \{\phi(x_i^s)\}$ and $\mathcal{D}_t = \{\phi(x_i^t)\}$ as the i -th hidden layer representations of the source and target domains, respectively. In the proposed DAN-based mechanism, the multi-layer penalty is applied only to the last two fully connected layers. The detail is depicted in Figure 3.

The feature extraction process generates outputs that are fed into the first fully connected layer, known as *fc1*. This stage yields the hidden representations for the first layer of both the source and target domains, represented by $\phi(x_i^{s1})$ and $\phi(x_i^{t1})$. These representations undergo processing through either the SCF-unNorm or SCF-pInverse functions, as outlined in Equations 5 and 6, which produce a metric termed the *transfer loss*.

Similarly, the second fully connected layer, *fc2*, receives $\phi(x_i^{s2})$ and $\phi(x_i^{t2})$, which are similarly subjected to SCF-unNorm or SCF-pInverse operations akin to *fc1*. Within the source domain, $\phi(x_i^{s2})$ is passed through the softmax function to yield the conditional probability $\theta(x_i^s)$, which is subsequently put into the cross-entropy loss function, as specified in Equation (12). The cumulative cross-entropy loss for the samples, along with the aggregate *transfer loss* from both *fc1* and *fc2*, are computed in Equation (13), and this result is utilized for the backpropagation process.

In contrast, the target domain differs in that $\phi(x_i^{t2})$ solely undergoes the softmax function, which computes the prediction probability for the unclassified application.

In Figure 3, the implicit features extracted from the input evolve from general to specific as they progress through the network. Consequently, the features derived from the convolutional layers of the feature extractor are general, warranting their freezing. Meanwhile, the fully-connected layers are customized to suit the new task, and thus, they are adapted with SCF. This adaptation process enables the DAN-based mechanism to learn implicit features from the source domain and effectively transfer them to the target domain, allowing it to address new tasks.

V. EXPERIMENTAL RESULTS

This section offers a detailed overview of the traffic classification dataset, covering aspects such as data characteristics, collection methodology, and partitioning. Additionally, it delineates the performance metrics and experimental configurations. The concluding segment of this section provides an extensive analysis of performance outcomes derived from three specific experiments: the classification of existing applications, new applications, and partially known applications.

A. Experimental Setups

1) *DAQUIC Dataset*: To evaluate the performance of our method and the benchmarks, we build a *DAQUIC* dataset, which is captured at two different periods. The first part was captured during three weeks in March 2018, including YouTube, Google Hangout Chat (Chat), and Google Hangout VoIP (VoIP). The second part was captured recently from March to April 2023, including video streaming of Facebook (called Facebook or FB for short) and commercial websites

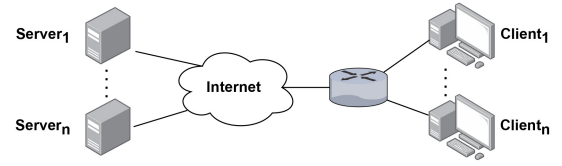


Fig. 4. The setup for the data collection.

in Southeast Asia such as Shopee, Tiki, Thegioididong, eBay, Amazon, and Alibaba. Various applications, such as Chat, VoIP, video streaming, and commercial websites, are considered.

These applications are considered due to two reasons:

- *Dominant applications in network traffic*: According to a recent report, e-commerce, video on-demand, and interactive data comprise over 80% of global IP traffic by 2022. Those 3 applications would represent the general cases of the real network traffic environment.
- *Domain adaptation requires a larger size of the source domain compared to the target domain, hence the selection of source and domain sets for testing is preferred to preserve this characteristic*. During the periods of March 2018 and March 2023, we gathered a substantial dataset comprising over 200GB of network traffic. Within this dataset, we identified approximately 20,000 flows across 10 different applications. The process of domain adaptation necessitates that the source domain contains a larger volume of data compared to the target domain. This requirement stems from our methodology of pre-training on the source domain and subsequently transferring the pre-trained models to classify categories within the target domain. To adhere to this principle, we divided the dataset into distinct source and target domains, as outlined in the manuscripts. It is essential to note that shuffling the applications randomly across both domains may inadvertently result in the target domain containing a greater volume of data than the source domain. Such an imbalance can lead to inaccuracies in our findings and analyses. Therefore, careful consideration must be given to maintaining the integrity of the source-target domain relationship to ensure the robustness and reliability of our results.

Setup: To collect the data, the testbed is set as in Figure 4. A client and a server are dedicated to collecting the data for an application (e.g., YouTube, Facebook, etc.). In the clients, selenium [36] tool is used to automate the connections between clients and servers. Regarding YouTube and Facebook, the clients watch a list of videos automatically. As for the e-commerce applications, the clients automate access to a list of links belonging to the e-commerce websites. For interactive applications (VoIP and chat), Google Hangout is used to automate access and generate network traffic. To make the data similar to real scenarios, a random duration (a few minutes) is set between two connections.

Data Collection: To collect the data of applications, tshark [37] is installed in each client. During the data collection, several noisy network flows containing only a few

TABLE I
THE NUMBER OF FLOWS IN THE TRAFFIC DATASET

Data sources	Applications	Nb of flows
Netflow-GQUIC	Chat	2556
	VoIP	2198
	Youtube	4645
Additional Applications	Shopee	2581
	Tiki	1205
	Thegiodidong	509
	Ebay	1836
	Amazon	834
	Alibaba	1401
	Facebook	1379

TABLE II
THE DATASET DESCRIPTION FOR SCENARIO *Data Sensitivity*

		E-commerce	Nb of flows	Video on-demand	Nb of flows	Interactive data	Nb of flows
Source		Tiki, Shopee, Thegiodidong	4295	Youtube	4645	Chat	2556
Target	Target 1	Ebay	1475	FB 1	356	VoIP 1	609
	Target 2	Amazon	1132	FB 2	387	VoIP 2	578
	Target 3	Alibaba	668	FB 3	376	VoIP 3	589

TABLE III
THE DATASET DESCRIPTION FOR SCENARIO *Parameter Sensitivity and for Partially Known Applications*

Categories	Source	Nb of flows	Target	Nb of flows
E-commerce	Tiki	4295	Ebay	4071
	Shopee		Amazon	
	Thegiodidong		Alibaba	
Video on-demand	Youtube	4645	Facebook	1379
Interactive data	Chat	2556	VoIP	2198
Total flows		11496		7648

packets were filtered out to avoid their influence. Regarding our data collection methodology, the collection process spans multiple sessions throughout a two-month period. Each session commences from the moment a Web browser is initiated using the Selenium tool for automated connections, continuing until the closure of the Web browser. It is important to note that different encryption keys are utilized across various sessions to encrypt data during transmission. As a result, the total count of session keys is significantly larger than initially conveyed. Overall, the DAQUIC dataset holds substantial potential for facilitating the generalization of the proposed approach.

Data Splitting: The number of flows for each application is shown in Table I, and the applications are grouped into three categories: e-commerce, video-on-demand, and interactive data. A category contains the applications which have similar data distribution. There are three scenarios in this paper. In the first scenario (data sensitivity), the data of the target domain is divided into three smaller sets (Table II) to evaluate the influence of data volume in the target domain on accuracy.

In the second scenario (parameter sensitivity), the data is separated into two small groups representing the source and target domains (Table III).

In the last scenario (for partially known applications), a small part of the data in the target domain is labeled and combined with the source domain data to evaluate the

TABLE IV
ACCURACY AND TESTING TIME OF DIFFERENT FEATURE SETS FOR TWO GROUPS

Feature Extractor	Nb of Features	Accuracy (%)	Avg. prediction time per flow (ms)
DarkNet53	32	82.96	2.55
	256	87.94	2.88
	512	88.92	3.47
Mobilevitv2_050	32	82.87	2.94
	256	87.30	3.21
	512	88.54	3.81
ResNet50	32	82.45	2.26
	256	88.24	2.56
	512	89.14	3.08

prediction performance on the target domain. The data of this scenario is derived from Table III.

2) *Performance Metrics:* Two key metrics are chosen to evaluate the performance of traffic classification mechanisms: accuracy and the average prediction time for a flow. The accuracy of the model reflects how well the model can correctly predict the outcome. A higher accuracy indicates that the model learns the data better and can make more reliable classifications. On the other hand, the average prediction time for a flow is an essential measure of the efficiency of a model. This metric provides insight into how quickly a model can process and predict outcomes for each data flow. A lower average prediction time signifies that a model is not only accurate but also efficient, making it highly suitable for real-time applications where speed is of the essence.

Regarding the byte-level features of a sample, the number of bytes k in the payload is set to 256 or 512, while the number of packets in a flow m is set to 20. In the payload-based DAN-based mechanisms, the fully-connected layer of the feature extractor is replaced by two fully-connected layers with 256 and 3 nodes, respectively. The learning rate and batch size are set to 0.0002 and 64, respectively. The stochastic gradient descent optimizer is used in these two mechanisms. These parameters are selected via the experiments. The implementation of the proposed method and benchmarks are written in Python using Pytorch and scikit-learn libraries. These mechanisms are implemented in a computer with Intel(R) Core i5-13400F @ 4.6 Ghz, 64GB of RAM, NVIDIA GeForce RTX 3080 10GB, and Ubuntu 20.04. The source code is available upon request.

B. Performance Analysis

1) *For Existing Applications:* In this subsection, the performance of the encrypted network traffic classification mechanism on the source domain is evaluated. The goal is to classify the traffic into different applications (e.g., tiki, shopee, thegiodidong, etc.). The dataset is divided into training, validation, and test sets using a 76:4:20 ratio. The experiment is conducted five times, and the results are averaged to obtain the values presented in Table IV.

Feature Extraction: Numerous feature extraction methods exist for traffic classification, but no standard one prevails. Three backbones are considered to be suitable feature extractors: ResNet50, DarkNet53, and Mobilevitv2_50. Among these options listed in Table IV, ResNet50 demonstrates the

TABLE V
ACCURACY (%) ON DIFFERENT TARGETS WITH 256 AND 512
FEATURES FOR THREE MECHANISMS

Nb of Features	Methods	MK-MMD	SCF-unNorm	SCF-pInverse
256	Target 1	53.57	66.84	57.48
	Target 2	44.15	46.98	49.19
	Target 3	34.35	42.75	52.67
512	Target 1	65.31	64.29	66.84
	Target 2	63.71	63.91	60.08
	Target 3	57.51	58.02	59.54

highest accuracy for both 256 and 512 features, achieving 89.14% accuracy with 512 features. Besides, ResNet50 also has the lowest prediction time for all feature sizes, taking only 2.26 ms with 32 features and 3.08 ms with 512 features. This indicates that ResNet50 is an appropriate solution in the considered backbones. The superiority of ResNet50 in both accuracy and prediction time can be attributed to its architectural design. ResNet50 utilises residual blocks which help mitigate the vanishing gradient problem, allowing it to learn more effectively from a large number of features. This is particularly beneficial in traffic classification tasks where subtle features can be crucial for accurate predictions. The prediction time is the average time required to classify a sample into one of three categories, excluding the time for collecting the input traffic and forming the input vector. Moreover, the lower prediction times of ResNet50 suggest that its layers are computationally less intensive, making it faster to process the input data. This is a significant advantage in real-time applications where quick decision-making is essential.

Feature Selection: The performance of the proposed mechanism and the benchmarks are evaluated with various byte values in the payload of a packet to select an appropriate one ($m \in (32, 256, 512)$). The results illustrate that increasing the number of features generally improves the accuracy of the models but also increases the prediction time. For instance, DarkNet53, with 32 features, has an accuracy of 82.96% and a prediction time of 2.55 ms, while it achieves an accuracy of 88.92% and requires a prediction time of 3.47 ms with 512 features. ResNet50 can achieve 88.24% accuracy and require 2.56 ms of prediction time with 256 features. When the number of features increases to 512, its accuracy increases to 89.14%, and it requires 3.08 ms of prediction time. The Mobilevitv2_50 backbones also have similar results. Depending on the requirements of traffic classification, an appropriate number of features should be selected to balance the trade-off of accuracy and prediction time.

2) *For New Applications:* In this subsection, the performance of the DAN-based mechanism for new applications is evaluated with various two-sample test statistics: SCF-unNorm, SCF-pInverse and MK-MMD. The experimental results are averaged after five times running with 250 epochs.

Data sensitivity: In this scenario, we evaluate the influence of the number of samples for the target domain on the accuracy of three considered mechanisms. We use target domain data from Table II and the source domain data in Table III. The objective of this scenario is to show the performance of DAN with varying amounts of data in the combined domain.

TABLE VI
CLASSIFICATION ACCURACY (%) WITH VARYING λ VALUES

Nb of features	Lambda	SCF-pInverse	SCF-unNorm	MK-MMD
256	0.05	91.72	90.32	89.00
	0.5	88.54	89.51	83.67
	1	90.51	83.37	87.98
	2	91.88	95.56	87.52
512	0.05	96.71	98.00	98.00
	0.5	95.09	97.69	90.78
	1	96.67	96.98	88.81
	2	98.39	96.87	90.46

Table V shows the results for 256 features. The accuracy decreases with the original data for all mechanisms. SCF-unNorm can achieve 66.84% accuracy for target 1, but it drops to 46.98% for target 2 and 42.75% for target 3. MK-MMD has the lowest accuracy for all targets, reaching only 34.35% for target 3. The accuracy of SCF-pInverse fluctuates slightly, decreasing from 57.48% for target 1 to 52.67% for target 3. These results show that a reduction in the number of samples leads to a decrease in the accuracy of traffic classification.

Table V also illustrates the results for 512 features. The accuracy is higher than that for 256 features due to the increase in the number of features, but it is still not optimal. All methods achieve over 60% accuracy for target 1, target 2, and target 3. Besides, SCF-unNorm and SCF-pInverse outperform MK-MMD in these targets. This indicates that an increase in features can partially compensate for the reduced sample size, though the accuracy remains less than ideal. The data patterns in the target domain are crucial for classification accuracy. A small sample size (ranging from 356 to 1475 for targets 1, 2, and 3) fails to bridge the domain discrepancy between the source and target domains. Specifically, the source and target domain features remain distinct even after applying DAN, leading to lower accuracy.

The results, as shown in Table V, demonstrate that the data patterns in the target domain correlate with the accuracy of traffic classification. When the number of samples is limited, the classification accuracy is compromised. The reason is that a small number of samples cannot adequately bridge the domain discrepancy. Specifically, the features of the source domain and the target domain remain distinct even after applying DAN, leading to lower accuracy. Therefore, subsequent scenarios will consider a larger number of samples in the target domain. The number of samples is described in Table III. The target domain in Table III is composed of target 1, 2, 3 in Table II while the source domain in the Table III and II is kept unchanged.

Parameter Sensitivity: Table VI aims to illustrate the accuracy of different mechanisms against various values of λ for two different numbers of features containing 256 and 512. The λ parameter controls the trade-off between the cross-entropy and the transfer loss in the domain adaptation process. The goal of this scenario is to select an appropriate λ value for the feature sets with 256 and 512 features.

We can observe that the sensitivity of the λ parameter varies for each mechanism and number of features. With 256 features, SCF-pInverse has an accuracy of 91.72% with

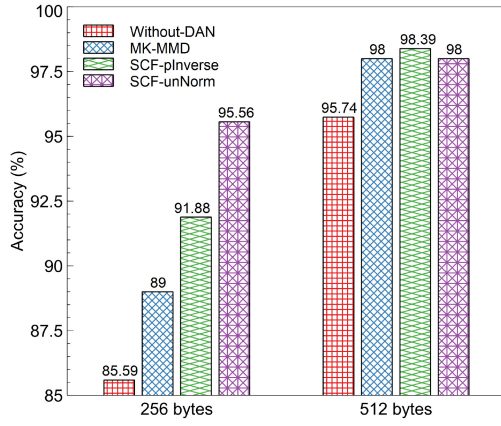


Fig. 5. The performance of the DAN-based mechanisms and the solution without DAN, which pre-trains on the source domain to obtain a pre-trained model and then directly uses this model to classify the new applications in the target domain into one of three categories.

$\lambda = 0.05$, but it drops to 88.54% with $\lambda = 0.5$. In contrast, SCF-unNorm has a low accuracy of 83.37% with $\lambda = 1$ for 256 features, and then its accuracy increases to 95.56% with $\lambda = 2$. On the other hand, MK-MMD has a relatively stable accuracy across different values of λ for both numbers of features, ranging from 83.67% to 89% for 256 features and from 88.81% to over 98% for 512 features.

Accuracy: Figure 5 aims to show that the proposed mechanisms (SCF-pInverse and SCF-unNorm) outperform the other mechanisms related to accuracy. In this figure, the highest accuracy of each mechanism is selected which is described in Table VI. With 256 and 512 features, the accuracy of SCF-unNorm and SCF-pInverse are higher than the others. Regarding 256 features, SCF-unNorm improves by nearly 10% compared to the solution without using DAN, respectively. As for 512 features, there is a slight difference between these mechanisms. SCF-unNorm only improves by nearly 3% in comparison with the solution without using DAN. Without using DAN, there is a large domain discrepancy between the source and the target domains, leading to inaccurate prediction. In contrast, using DAN (SCF-unNorm, SCF-pInverse and MK-MMD) reduces the domain discrepancy between these domains, enhancing the accuracy. Moreover, with 256 features, SCF-unNorm improves by nearly 6% compared to MK-MMD. However, with 512 features, there is no difference between these mechanisms. The reason is that MK-MMD uses the smooth witness function, which does not maximize the test power, while SCF uses the analytic representations of probability distributions, which can maximize the power test better. Besides, using 512 features can achieve better accuracy than using 256 features. However, the prediction time for 512 features is slightly higher than the figure for 256 features.

Training time: The training time of different mechanisms for 256 and 512 features is depicted in Figure 6. This scenario aims to illustrate that the proposed mechanism requires less training time compared to MK-MMD. Regarding the training time of MK-MMD, the linear formula is used to measure the domain discrepancy in deep adaptive networks [9]. However, the training time of MK-MMD is higher than SCF-unNorm

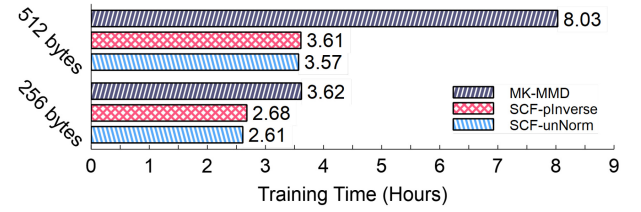


Fig. 6. Training time of MK-MMD, SCF-unNorm, SCF-pInverse with 256 and 512 features.

and SCF-pInverse. Concretely, with 256 features, MK-MMD requires 3.62 hours to complete 250 epochs of training, while SCF-unNorm and SCF-pInverse only require 2.61 and 2.68 hours to complete their training phase. Therefore, MK-MMD takes one hour longer than SCF-unNorm (fastest mechanism). This difference is even more evident with 512 features (Figure 6). MK-MMD takes more than 8 hours to complete its training, while SCF-unNorm and SCF-pInverse require 3.51 and 3.61 hours, respectively. By increasing from 256 to 512 features, the difference in training time between MMD and SCF-unNorm increases from just over 1 hour to 4.53 hours. In other words, SCF-unNorm can reduce 27.9% of training time with 256 features and 55.5% of training time with 512 features compared to MK-MMD. The reason is that SCF-unNorm uses a simple linear combination of multiple kernels while MK-MMD optimizes a complex objective function with multiple constraints. When the number of features increases to more than 512 (e.g., 1024, 1400, etc.), MK-MMD will reveal its drawbacks related to the time complexity more clearly. Besides, the training of SCF-unNorm is slightly lower than the figure for SCF-pInverse. The reason is that SCF-unNorm does not need to calculate the matrix S_n^+ as in SCF-pInverse.

Benchmarks: The performance of the proposed mechanism is evaluated with recent studies using the payload as the input with $\lambda = 1$: FixMatch [28], MCC [29], Binary-Zen-tor and Multi-Class-Zen-tor [26] (in Table VII). The goal is to show the comparison of the proposed approach and the recent studies. FixMatch generates pseudo-labels for the target data using the prediction model and then trains another model using pseudo-labels to predict on strongly-augmented samples, while MCC is a general loss function that reduces the confusion between the correct and ambiguous classes for the target data. Binary-Zen-tor is a GAN-based binary classification approach which classifies network traffic into two classes: existing applications and new ones. Multi-Class-Zen-tor is a GAN-based multiclass classification approach which not only performs binary classification like Binary-Zen-tor but also identifies different kinds of existing applications. Table VII shows the accuracy of these mechanisms. Using DAN with SCF function, DAN-pInverse can learn an adequate representation of the source domain and transfer it to the target domain. Therefore, the accuracy of DAN-pInverse is better than others, achieving 90.51% accuracy. In contrast, FixMatch generates the pseudo-labels for the unlabeled data, but the data labeling is not good, influencing its performance (only achieving 76.12% accuracy). MCC is a general loss function that is not adapted to the traffic classification. As a result, its

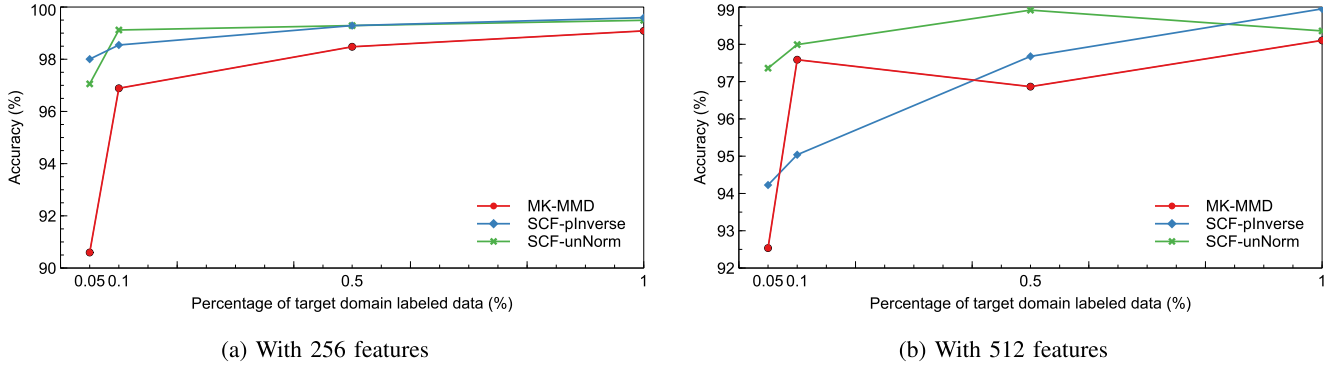


Fig. 7. Accuracy of MK-MMD, SCF-unNorm, SCF-pInverse methods with few labeled samples in the target domain.

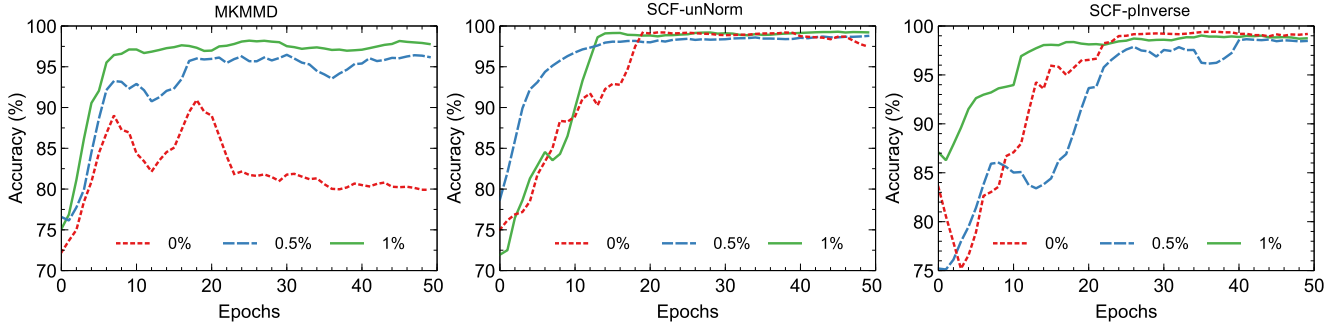


Fig. 8. The accuracy against the number of epochs with 0%, 0.5%, 1% of labeled data in the target data with 256 features.

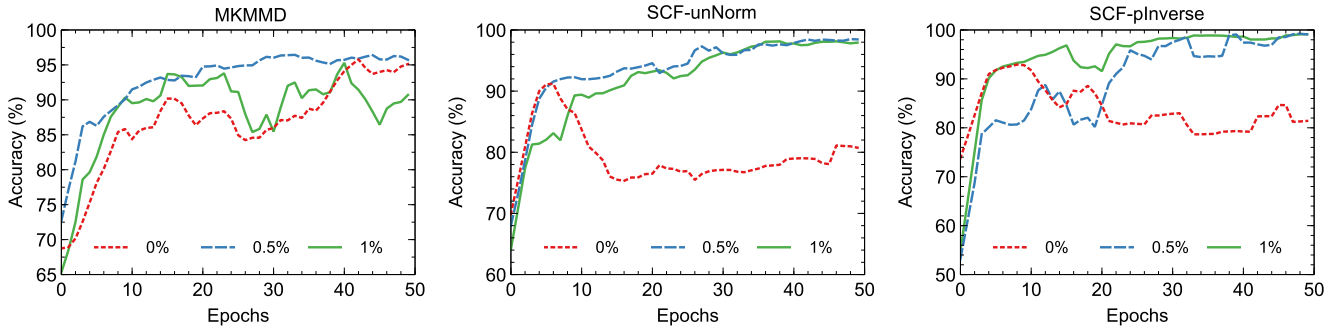


Fig. 9. The accuracy against the number of epochs with 0%, 0.5%, 1% of labeled data in the target data with 512 features.

accuracy is approximately 62%. Binary-Zen-tor achieves only 36.94% accuracy, while Multi-Class-Zen-tor performs better with 72.94% accuracy. The discriminator in Binary-Zen-tor classifies traffic into two categories: existing applications or new ones. In contrast, the discriminator in Multi-Class-Zen-tor performs two tasks. The first task is classifying traffic as either from existing applications or new ones, and the second task is further classifying existing applications into specific categories. Multi-Class-Zen-tor incorporates additional loss functions for the second task, which results in improved performance.

3) *For Partially Known Applications*: This subsection is dedicated to the scenario where a small fraction of the data in the target domain is labeled. The source and target domains are depicted in Table III. The proportions of labeled data considered are 0.05, 0.1, 0.5, and 1%. The details are described in Figure 7(a) and 7(b). This scenario aims to show that a small amount of labeled data in the target domain can enhance

TABLE VII
ACCURACY (%) OF THE PROPOSED MECHANISM AND BENCHMARKS

Methods	Accuracy
FixMatch [28]	76.12
Minimum Class Confusion (MCC) [29]	62.45
Binary-Zen-tor [26]	36.94
Multi-Class-Zen-tor [26]	72.94
Proposed Mechanism (DAN-pInverse)	90.51

the performance of the DAN-based mechanism, achieving up to over 99% accuracy. There is an upward trend in the accuracy against the number of labeled data for both 256 and 512 features. The reason is that a higher number of labeled data makes the training using DAN more effective. The domain discrepancy is reduced, and the accuracy improves better than without using labeled data.

For 256 features [see Figure 7(a)], the accuracy of MK-MMD increases from 90.6% at 0.05% to 96.89% at 0.1% of labeled data, then gradually rises to 99.09% at 1%. A noticeable observation is that SCF-pInverse and SCF-unNorm outperform MK-MMD in the considered percentage of labeled data. For 512 features [see Figure 7(b)], the accuracy of SCF-pInverse and SCF-unNorm is also higher than the figure for MK-MMD. However, there is a slight fluctuation in the accuracy of MK-MMD. In addition, SCF-pInverse with 512 features exhibits slightly lower accuracy than the 256-feature model. This is likely due to the increased noise introduced by the additional labeled data, which can negatively impact the model's performance.

Figure 8 and 9 aim to illustrate the accuracy of three mechanisms against the number of epochs for 256 and 512 features. The accuracy of using 1% of labeled data is higher than the figure without using labeled data for both 256 and 512 features. In addition to higher accuracy, using a small percentage of labeled data helps to reach high accuracy faster than unlabeled data. Regarding 256 features, all three mechanisms can converge to optimal state within from 10 to 15 epochs. Without using labeled data, it is required from 20 to 25 epochs. It is similar to 512 features.

C. Discussion

Regarding the payload-based mechanisms (Table IV), the accuracy increases against the number of features. However, the prediction time per flow (sample) also goes up when the number of features increases from 32 to 512. In three considered backbones, ResNet50 can obtain 88.24% accuracy while requiring a suitable prediction time of 2.56 ms.

As for the DAN-based mechanisms, the accuracy of MK-MMD, SCF-unNorm, and SCF-pInverse are not good (Table V) when the number of samples in the target domain is limited, as shown in Table II. When the number of samples in the target domain increases, as shown in Table III, the accuracy of these three functions is higher. SCF-pInverse and SCF-unNorm can achieve up to 98% of accuracy with 512 features. When adding a few percent of labeled data to the dataset in Table III, the performance of MK-MMD, SCF-unNorm and SCF-pInverse are improved significantly, as in Fig. 7. There is an upward trend in the accuracy of these functions when the percentage of labeled data increases from 0.05 to 1 percent. The accuracy of SCF-unNorm and SCF-pInverse can achieve up to approximately 99% for both 256 and 512 features.

Despite obtaining good performance, the proposed DAN-based mechanisms (SCF-unNorm and SCF-pInverse) contain several limitations. First, the influence of the transfer loss in the target risk relies on the λ parameter. When λ is small, the improvement of the accuracy is small. Otherwise, the improvement of the accuracy is higher, but the deviation of the accuracy between running times is large. Second, the proposed DAN-based approaches are effective under the assumption that the source domain is related to the target domain. For example, the new applications in the target domain (e.g., eBay, etc.) are different from existing applications in the source domain (e.g.,

shopee, etc.), but the data distribution of these applications are related.

Moreover, the performance metrics are produced in the considered scenarios along with considered parameters in the experimental results. Besides, the proposed DAN-based mechanisms are designed for three categories: e-commerce, video on-demand, interactive data. Implementing the proposed mechanisms with other categories and different parameter settings can lead to the results that are not reported in this research.

VI. CONCLUSION

In this paper, we propose a Deep Adaptation Network using novel Smooth Characteristic Functions (**SCF-unNorm** and **SCF-pInverse**) to enhance the performance of encrypted traffic classification. The experimental results show that the proposed mechanism achieves up to 99% accuracy and requires an appropriate processing time (only 3.08 ms) for 512 features (512 byte values in the payload of packets). Moreover, to evaluate the performance of this work and benchmarks, the traffic classification dataset for domain adaptation is built and provided publicly for the research communities. The proposed mechanism using the Deep Adaptation Network is dedicated to identifying new applications that are not trained prior, while the proposed payload-based encrypted traffic classification mechanism aims to identify existing applications that are trained prior. Additionally, we plan to incorporate a mechanism for classifying network flows into existing or new applications and integrate it into our framework to create a complete system. Furthermore, the proposed approach aims to classify encrypted traffic into different categories (e.g., e-commerce, etc.) with limited labeled data. In the future, we will enhance the proposed approach to classify traffic into specific applications (e.g., Facebook, YouTube, etc.)

REFERENCES

- [1] Z. Li et al., "Problem statement and use cases of application-aware networking (APN)," Internet Eng. Task Force, Internet-Draft draft-ietf-spring-segment-routing-16, 2020.
- [2] V. Tong, S. Souihi, H. A. Tran, and A. Mellouk, "SDN-based application-aware segment routing for large-scale network," *IEEE Syst. J.*, vol. 16, no. 3, pp. 4401–4410, Sep. 2022.
- [3] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.
- [4] Y. Yang et al., "A network traffic classification method based on dual-mode feature extraction and hybrid neural networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4073–4084, Dec. 2023.
- [5] V. Tong, H. A. Tran, S. Souihi, and A. Mellouk, "A novel QUIC traffic classifier based on convolutional neural networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2018, pp. 1–6.
- [6] H. Tahaei, F. Afifi, A. Asemi, F. Zaki, and N. B. Anuar, "The rise of traffic classification in IoT networks: A survey," *J. Netw. Comput. Appl.*, vol. 154, Mar. 2020, Art. no. 102538.
- [7] P. Wang, F. Ye, X. Chen, and Y. Qian, "Datanet: Deep learning based encrypted network traffic classification in SDN home gateway," *IEEE Access*, vol. 6, pp. 55380–55391, 2018.
- [8] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.
- [9] M. Long, Y. Cao, Z. Cao, J. Wang, and M. I. Jordan, "Transferable representation learning with deep adaptation networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 12, pp. 3071–3085, Dec. 2018.

- [10] M. Long, Y. Cao, J. Wang, and M. Jordan, "Learning transferable features with deep adaptation networks," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 97–105.
- [11] F. Zhuang et al., "A comprehensive survey on transfer learning," *Proc. IEEE*, vol. 109, no. 1, pp. 43–76, Jan. 2021.
- [12] J. Jiang, Y. Shu, J. Wang, and M. Long, "Transferability in deep learning: A survey," 2022, *arXiv:2201.05867*.
- [13] Z. Taghiyarrenani and H. Farsi, "Domain adaptation with maximum margin criterion with application to network traffic classification," in *Machine Learning and Principles of Knowledge Discovery in Databases*. Grenoble, France: Springer, 2022, pp. 159–169.
- [14] J. Ning et al., "Malware traffic classification using domain adaptation and ladder network for secure Industrial Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 17058–17069, Sep. 2022.
- [15] W. Jitkrittum, Z. Szabó, K. P. Chwialkowski, and A. Gretton, "Interpretable distribution features with maximum testing power," in *Proc. 30th Conf. Neural Inf. Process. Syst.*, vol. 29, 2016, pp. 1–9.
- [16] C. Dao, V. Tong, N.-T. Hoang, H.-A. Tran, and T. X. Tran, "Enhancing encrypted traffic classification with deep adaptation networks," in *Proc. IEEE 48th Conf. Local Comput. Netw. (LCN)*, 2023, pp. 1–4.
- [17] V. Tong and C. Dao, "Traffic classification dataset for deep adaptation network." Nov. 2023. [Online]. Available: <https://github.com/vanvantong/Encrypted-Traffic-Classification-using-DAN>
- [18] Y. Hu, G. Cheng, W. Chen, and B. Jiang, "Attribute-based zero-shot learning for encrypted traffic classification," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4583–4599, Dec. 2022.
- [19] S. Cui, J. Liu, C. Dong, Z. Lu, and D. Du, "Only header: A reliable encrypted traffic classification framework without privacy risk," *Soft Comput.*, vol. 26, no. 24, pp. 13391–13403, 2022.
- [20] H. He, Y. Lai, Y. Wang, S. Le, and Z. Zhao, "A data skew-based unknown traffic classification approach for TLS applications," *Future Gener. Comput. Syst.*, vol. 138, pp. 1–12, Jan. 2023.
- [21] S. Fathi-Kazerooni, Y. Kaymak, and R. Rojas-Cessa, "Tracking user application activity by using machine learning techniques on network traffic," in *Proc. Int. Conf. Artif. Intell. Inf. Commun. (ICAIIIC)*, 2019, pp. 405–410.
- [22] M. Qin, K. Lei, B. Bai, and G. Zhang, "Towards a profiling view for unsupervised traffic classification by exploring the statistic features and link patterns," in *Proc. Workshop Netw. Meets AI ML*, 2019, pp. 50–56.
- [23] M. S. Raza, K. A. Bhatti, F. M. Malik, and S. A. Sheikh, "Network traffic classification using deep neural networks," in *Proc. Int. Conf. Front. Inf. Technol. (FIT)*, 2023, pp. 85–89.
- [24] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Security Inform. (ISI)*, 2017, pp. 43–48.
- [25] Y. Li et al., "Federated domain Generalization: A survey," 2023, *arXiv:2306.01334*.
- [26] Y. Gu, Y. Lai, and Y. Wang, "Zen-tor: A zero knowledge known-unknown traffic classification method," in *Proc. GLOBECOM IEEE Global Commun. Conf.*, 2022, pp. 885–890.
- [27] S. K. Nukavarapu, M. Ayyat, and T. Nadeem, "MirageNet-towards a GAN-based framework for synthetic network traffic generation," in *Proc. GLOBECOM IEEE Global Commun. Conf.*, 2022, pp. 3089–3095.
- [28] K. Sohn et al., "Fixmatch: Simplifying semi-supervised learning with consistency and confidence," in *Proc. 34th Conf. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 596–608.
- [29] Y. Jin, X. Wang, M. Long, and J. Wang, "Minimum class confusion for versatile domain adaptation," in *Proc. 16th Eur. Conf. Comput. Vis. (ECCV)*, Glasgow, U.K., 2020, pp. 464–480.
- [30] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, "A theory of learning from different domains," *Mach. Learn.*, vol. 79, pp. 151–175, May 2010.
- [31] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, "A kernel two-sample test," *J. Mach. Learn. Res.*, vol. 13, no. 1, pp. 723–773, 2012.
- [32] P. Tang and A. Xu, "Generalized Leibniz functional matrices and divided difference form of the Lagrange–Bürmann formula," *Linear Algebra Appl.*, vol. 436, no. 3, pp. 618–630, 2012.
- [33] K. P. Chwialkowski, A. Ramdas, D. Sejdinovic, and A. Gretton, "Fast two-sample testing with analytic representations of probability measures," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 1–9.
- [34] E. H. Moore, "On the reciprocal of the general algebraic matrix," *Bull. Am. Math. Soc.*, vol. 26, no. 9, pp. 394–395, 1920.
- [35] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Network traffic classifier with convolutional and recurrent neural networks for Internet of Things," *IEEE Access*, vol. 5, pp. 18042–18050, 2017.
- [36] B. García, M. Muñoz-Organero, C. Alario-Hoyos, and C. D. Kloos, "Automated driver management for selenium WebDriver," *Empirical Softw. Eng.*, vol. 26, pp. 1–51, Jul. 2021.
- [37] S. Qureshi, S. Tunio, F. Akhtar, A. Wajahat, A. Nazir, and F. Ullah, "Network forensics: A comprehensive review of tools and techniques," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 5, pp. 879–887, 2021.